

TTFL: Dual-Timescale Trust Flow for TEE-Anchored Federated Learning under Non-IID Data

Anonymous Author(s)

Abstract. Federated learning (FL) enables collaborative edge intelligence without moving raw data, but open participation creates a trust-continuity problem: a client may execute an admissible training process while producing a harmful update, and a benign long-tail client may look anomalous under Non-IID data. This paper presents TTFL as a trust-flow framework that assigns different evidence to different decisions. Execution evidence establishes an admission boundary, privacy-aware data and runtime checks provide limited statistical evidence, and the cloud server audits update content and cross-round behavior before assigning layer privileges. The central design is a temporal separation between long-term utility and short-term risk: a persistent contribution signal supports retention, whereas a current risk signal can restrict an update without immediately erasing a client’s history. A conservative cold-start policy, explicit state transitions, and risk-gated data-size-aware re-normalization complete the decision loop. The evaluation uses an Intel TDX-backed client execution path with a paired host configuration, Flower/PyTorch and CIFAR-10, five seeds per configuration, 20 admitted clients, six attack behaviors, and a 30% malicious setting. Targeted ASR is macro-averaged over the three targeted behaviors, while untargeted attacks are evaluated with clean accuracy, loss, and state-transition metrics. TTFL reaches 92.31% clean accuracy, 10.21% targeted ASR under the stated protocol, and no permanent false ban in the tested runs.

Keywords: Federated Learning · Edge Computing · Trusted Execution Environment · Risk Modeling · Layered Aggregation · Backdoor Defense

1 Introduction

FL enables edge devices to train a shared model through parameter exchange while keeping their raw samples local [1,2]. In an open edge deployment, however, the server must assess both the execution that produced an update and its effect on the global model. This assessment becomes difficult under Non-IID data: a legitimate long-tail client may repeatedly deviate from a reference direction, whereas an attacker may build a favorable record before submitting a harmful update. A single reputation score makes these cases difficult to separate. Tightening its threshold can remove useful heterogeneous contributions, while allowing past performance to compensate for current anomalies can delay the response to an attack. The resulting challenge is to preserve evidence of long-term contribution without allowing that evidence to conceal short-term risk.

Existing defenses use geometric, statistical, reference-based, or temporal evidence to flag unusual updates [3,4,5,6,8]. Backdoor-oriented methods additionally examine model behavior, learned representations, clusters, and collusion [9,10,12,13]. Such tests say little about how local training was executed, and the same tests can interpret Non-IID drift as an attack. TEE-assisted FL contributes a hardware-rooted execution boundary [15,16,18,19]; admission based on that boundary still leaves the semantic safety of an admitted update unresolved.

The proposed *Trust Flow* policy addresses this tension by maintaining historical utility and short-term risk as separate states. Historical utility governs how a client retains or recovers its contribution, while current risk can restrict an update despite a favorable earlier record. These states connect execution admission, content auditing, and layer-wise aggregation: Intel TDX establishes the execution boundary, current evidence determines the submitted update’s influence, and persistence rules govern subsequent participation. Assigning evidence to these different decisions allows the framework to distinguish a temporarily unhelpful client from one that presents sustained risk. The contributions are:

1. A dual-stream state representation gives historical contribution and short-term risk separate roles, preserving benign-client retention while allowing a rapid response to attacks.
2. A state-to-aggregation policy links recoverable suspicion, persistent-risk escalation, and layer-specific weighting. Content quality can lower a contribution without, by itself, forcing irreversible exclusion.
3. An empirical analysis relates the single-stream and HistPerf-only ablations to the retention–detection tradeoff, and separates the effects of risk probes from those of weight re-normalization.

2 Background and Related Work

Federated learning in edge computing. Local training and server aggregation allow clients to collaborate without transferring raw samples, but leave the server dependent on the evidence accompanying each update. Heterogeneous data distributions complicate that assessment because a useful long-tail contribution may differ from the majority. Execution provenance and update quality consequently serve different purposes: the former establishes how an update was produced, while the latter determines how it should contribute to learning.

Poisoning and backdoor attacks. Sign flipping and gradient scaling disrupt optimization by changing update directions or magnitudes. Trigger, clean-label, and semantic backdoors instead seek targeted errors while preserving plausible main-task behavior, so their updates can remain close to an expected direction [20,21]. Across rounds, an attacker may also accumulate a favorable history before injecting a harmful update, making current behavior and past contribution potentially conflicting sources of evidence.

Active defenses and their limits. Krum and Trimmed Mean remove geometric outliers [3,4], FLTrust weights updates against a clean direction [5],

and FoolsGold and FLDetector use cross-client or cross-round evidence [6,8]. DeepSight, FLAME, CrowdGuard, RoseAgg, FLPurifier, ShieldFL, and RaSA inspect representations, backdoor behavior, collusion, or privacy-preserving updates [9,10,12,13,11,14,7]. TMT-FL and related systems connect execution integrity to model training [19]. These approaches observe different evidence spaces. Trust Flow places an interface between them: attestation determines execution eligibility, privacy-aware checks expose bounded statistics, content probes inspect the current update, and the two temporal streams decide future privileges. One observation can therefore alter a layer weight, a recoverable state, or a persistent state only when the corresponding rule is satisfied. The experiments use classical methods as matched aggregation baselines and stateful variants to isolate this temporal interface.

TEE-assisted trusted training. TEEs protect execution and key material and have been used to support FL integrity and attack mitigation [15,16,17,18,19]. A quote authenticates the measured path, freshness, and update provenance. TTFL combines that hardware evidence with privacy-aware verification and cloud-side semantic auditing before assigning aggregation privileges.

3 System Model and Threat Assumptions

Figure 1 presents the client-edge-cloud environment, consisting of a central aggregation server $\mathcal{S}$ and K edge clients $\mathcal{C} = \{c_1, \dots, c_K\}$. In round t , admitted clients train from the distributed model $W^{(t-1)}$ and submit updates $\Delta W_k^{(t)}$ for auditing and aggregation. The server then computes $W^{(t)} = W^{(t-1)} + \text{Agg}(\{\Delta W_k^{(t)}\})$, with the global learning objective

$$\min_W \mathcal{L}(W) = \sum_{k=1}^K p_k \mathcal{L}_k(W), \quad p_k = |\mathcal{D}_k| / \sum_j |\mathcal{D}_j|. \quad (1)$$

The client side performs local training and supplies an attestation-bound report, while privacy-aware verification exposes the declared data and runtime statistics needed for admission and audit. Raw samples remain local. Using the submitted update and this evidence, the cloud server evolves the client’s trust states and determines its aggregation privilege. The division of responsibilities connects execution verification to model-level control: admission establishes the provenance of an update, and subsequent auditing determines its contribution. Because these decisions require individual updates and audit evidence, the workflow does not use cryptographic secure aggregation to hide them from the server.

Threat model. Figure 2 summarizes the attack surface. The server is honest-but-curious and follows the aggregation protocol. It can see individual updates and audit statistics, but it does not change the stated state-transition rules. An admitted client may alter labels, samples, or optimization objectives and may conduct label flipping, trigger backdoors, clean-label poisoning, semantic backdoors, sign flipping, or gradient scaling.

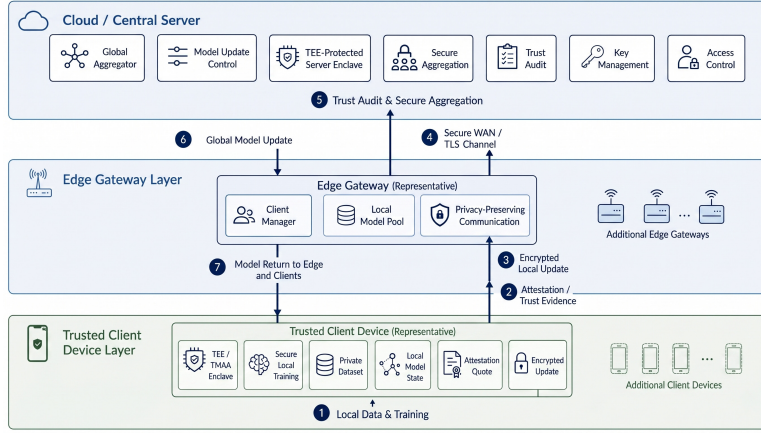


Fig. 1. TEE-anchored client-edge-cloud architecture.

In the deployed Intel TDX path, the client runs inside a measured confidential virtual machine. Its TDX evidence binds a verifier nonce, round identifier, and update digest to the measured workload. The boundary protects the measured path, device keys, report integrity and freshness, and the update binding against a compromised host. Privacy-aware verification and cloud-side content and temporal audits complement this hardware evidence by checking selected declared statistics and model-level behavior without exposing raw samples. Data- or objective-level poisoning through a valid measured process remains in scope; host tampering after attestation does not. A failed report or update-binding check is rejected during admission, after which valid evidence enters the content and temporal stages. Physical probing and advanced microarchitectural side channels are excluded. The analytical assumption is fewer than 50% malicious admitted clients; the 50% experiment is an out-of-assumption stress test, not a Byzantine threshold or a theoretical guarantee. The server-held clean proxy initializes content auditing, while the trusted-peer branch provides the corresponding operation when no proxy is available.

Design goals. Trust Flow is designed to maintain convergence under the stated attacks, avoid permanent false bans of long-tail clients, and keep the assurance boundary explicit. Execution, current content, historical utility, and recent risk retain separate meanings until layer-wise aggregation. The reported TrustReport size is about 15 KB, and server aggregation plus audit increases from 2.10 s to 4.85 s. The auxiliary security path measures approximately 11 ms per admitted-client round, while device-side TEE Quote generation, signing, and lightweight monitoring measure approximately 12–15 ms per admitted-client round. These component measurements are reported separately from server processing and end-to-end communication.

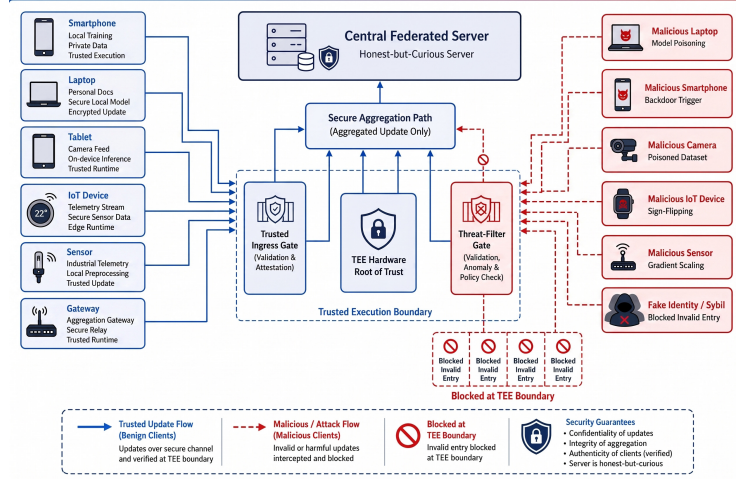


Fig. 2. Composite poisoning and penetration threat model.

4 Trust-Flow Framework

Each FL round passes through four linked stages: admission and runtime assessment, reference-based content auditing, dual-stream state evolution, and risk-gated layered aggregation. Figure 3 presents the loop.

The stages in Fig. 3 pass progressively different evidence to the aggregation decision. Admission and runtime assessment produce *TrustScore*, content auditing produces *ContentScore*, and state evolution accumulates this evidence in *HistPerf* and *RiskEMA*. Their combination yields *RawScore*, which controls layer privileges. This ordering allows the server to reject an invalid proof before inspecting update content, then distinguish a transient observation from persistent behavior before assigning aggregation weights. Cold start follows the same logic: current evidence determines a provisional first-round privilege, while irreversible transitions require accumulated evidence. A benign low-similarity update can therefore reduce influence without immediately ending the client’s opportunity to recover.

4.1 Admission and Content Auditing

Admission begins with a TEE check. Let $M_{\text{attest},k} \in \{0,1\}$ indicate whether the report, freshness nonce, and binding to the round and update digest are valid. During initialization, the client invokes a device-unique key and creates a remote-attestation quote containing the vendor measurement digest, code identity, runtime configuration, round nonce, update digest, and monotonic freshness counter. Communication privileges are granted only after this check. During local training, the trusted management agent records the standardized sequence $\mathcal{X}_k =$

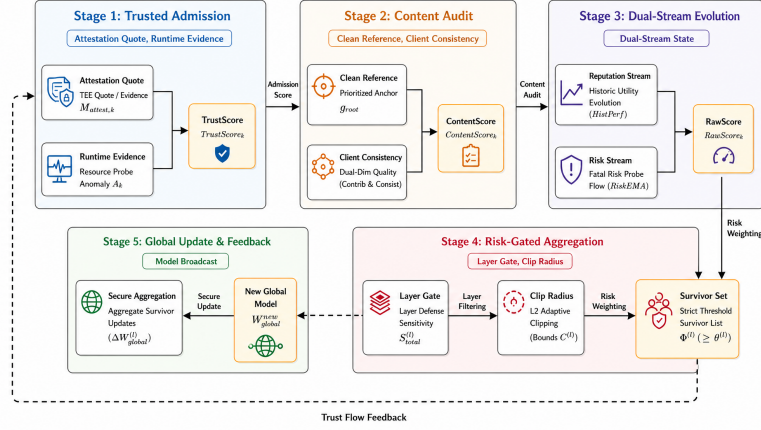


Fig. 3. Overview of the four-stage trust-flow defense loop.

$\{\Delta\text{grad}, \Delta\text{loss}, \text{instr_dist}\}$. The normalized entropy $h_k^{(t)} = H(\mathcal{X}_k^{(t)})/H_{\max} \in [0, 1]$ is computed after concatenating the three standardized feature histograms; $H_{\max}$ denotes the entropy of their uniform concatenation. The implementation uses

$$R_k^{(t)} = \exp(\lambda h_k^{(t)}), \quad Z_k^{(t)} = \text{clip}\left(w_a M_{\text{attest},k} + (1 - w_a)(1 - h_k^{(t)}), 0, 1\right), \quad (2)$$

where $w_a = 0.7$ and $\lambda = 1.0$. High entropy is therefore treated as uncertain evidence, not as a direct maliciousness label.

The trust state follows a scalar Kalman filtering view. It is updated as

$$T_k^{-(t)} = T_k^{(t-1)}, \quad P_k^{-(t)} = P_k^{(t-1)} + Q, \quad (3)$$

$$\begin{aligned} K_k^{(t)} &= \frac{P_k^{-(t)}}{P_k^{-(t)} + R_k^{(t)}}, \\ T_k^{(t)} &= T_k^{-(t)} + K_k^{(t)}(Z_k^{(t)} - T_k^{-(t)}), \\ P_k^{(t)} &= (1 - K_k^{(t)})P_k^{-(t)}. \end{aligned} \quad (4)$$

The observation $Z_k^{(t)} \in [0, 1]$ is defined above. Here $Q = 0.01$ is the fixed process-noise variance and $R_k^{(t)}$ is the entropy-derived observation variance. A fresh admitted client starts with $T_k^{(0)} = 1$, $P_k^{(0)} = 0.25$, $RiskEMA_k^{(0)} = 0$, and $M_{\text{attest},k} = 1$. The resulting privilege factor is

$$TrustScore_k^{(t)} = M_{\text{attest},k} \text{clip}\left(\frac{T_k^{(t)}}{1 + P_k^{(t)}}, 0, 1\right). \quad (5)$$

High entropy increases $R_k^{(t)}$ and lowers the Kalman gain, reducing the influence of an uncertain observation on the cross-round trust estimate. The covariance

discount in *TrustScore* also reduces privilege when confidence is low. Together, these operations allow runtime variability to affect participation without treating every fluctuation as evidence of an attack.

The server next constructs a clean guiding direction. When a small clean validation gradient exists, $g_{root} = g_{root_clean}$. Otherwise, admitted clients with high trust and low previous risk are used with reference weights $\omega_k^{ref} = TrustScore_k^{(t)}(1 - RiskEMA_k^{(t-1)})^2$. In the reported experiments, the server holds a balanced 500-sample proxy set, with 50 images per CIFAR-10 class, and this set is excluded from client training. The fixed proxy partition is part of the common experiment protocol. Methods with a reference channel use it for their defined reference operation, whereas methods without that channel retain their own aggregation rule under the same data, initialization, and communication budget. The proxy gives content auditing a stable starting direction. Peer consistency and temporal states continue to add evidence after initialization. If the proxy is unavailable, the trusted-peer branch builds the corresponding reference from admitted updates. For each update, content quality combines reference consistency and peer consistency:

$$S_{contrib,k} = \max(0, \alpha_1 + \alpha_2 \cos(\Delta W_k, g_{root})), \quad (6)$$

$$S_{consist,k} = \frac{\sum_{j \neq k} TrustScore_j \max(0, \alpha_1 + \alpha_2 \cos(\Delta W_k, \Delta W_j))}{\sum_{j \neq k} TrustScore_j + \varepsilon}, \quad (7)$$

$$ContentScore_k = \frac{(1 + \beta_{fusion}^2) S_{consist,k} S_{contrib,k}}{\beta_{fusion}^2 S_{consist,k} + S_{contrib,k} + \varepsilon}. \quad (8)$$

The harmonic fusion requires both reference consistency and peer consistency, with $\beta_{fusion} > 1$ placing greater emphasis on the clean direction. This combination addresses different ambiguities in the two signals: agreement among clients can reflect collusion, whereas disagreement with a clean direction can arise from legitimate Non-IID data. The resulting score measures current contribution quality; the temporal stage determines how that observation should affect continued participation.

4.2 Dual-Stream State Evolution

A low content score can reflect either a useful client's skewed data or a harmful update. Since those cases require different participation decisions, Trust Flow retains historical utility and short-term risk separately instead of making one reputation value carry both meanings. Let $q_k^{(t)} = \text{clip}(ContentScore_k^{(t)}, 0, 1)$. The historical stream accumulates this quality evidence through an explicitly decayed Beta model:

$$a_k^{(t)} = \beta_h a_k^{(t-1)} + q_k^{(t)}, \quad b_k^{(t)} = \beta_h b_k^{(t-1)} + 1 - q_k^{(t)}, \quad HistPerf_k^{(t)} = \frac{a_k^{(t)}}{a_k^{(t)} + b_k^{(t)} + \varepsilon}. \quad (9)$$

For a fresh client, $a_k^{(0)} = b_k^{(0)} = 1$, so $HistPerf_k^{(0)} = 0.5$ rather than a presumed perfect reputation. The decay factor $\beta_h \in (0, 1)$ is fixed before test rounds. In the first round, a validly admitted client stays in NORMAL and receives only a provisional privilege based on current content and risk. That evidence is stored for the next state update; it cannot alone trigger an irreversible blacklist. Immediate control is therefore available through provisional weighting, while permanent decisions wait for cross-round evidence.

The history channel changes slowly so that a few weak contributions do not erase a client's accumulated utility. For a legitimate long-tail client, low similarity can lower aggregation privilege while leaving later rounds available for recovery. The risk channel instead retains recent warnings through $RiskEMA_k^{(t)} = \beta_r RiskEMA_k^{(t-1)} + (1 - \beta_r) Risk_k^{inst}$, where $Risk_k^{inst}$ is the maximum current probe. Taking the maximum preserves a strong warning across probe channels; maintaining risk separately prevents a favorable history from altering that risk estimate. The state-transition rules then determine whether the evidence warrants review, quarantine, or permanent exclusion.

Let $u_k^{(t)} = 1 - HistPerf_k^{(t)}$ and $r_k^{(t)} = RiskEMA_k^{(t)}$. The thresholds $(\tau_h, \tau_s, \tau_q, \tau_b) = (0.26, 0.64, 0.80, 0.90)$ govern utility deficit, soft risk, quarantine, and blacklisting. A NORMAL client moves to SUSPECT after two consecutive rounds with $u_k^{(t)} > \tau_h$ or $r_k^{(t)} \geq \tau_s$. From SUSPECT, three such rounds or one current probe with $Risk_k^{inst} \geq \tau_q$ cause QUARANTINE. A QUARANTINE client enters BLACKLIST after four consecutive rounds with $r_k^{(t)} \geq \tau_b$. SUSPECT and QUARANTINE recovery requires two consecutive rounds satisfying $u_k^{(t)} \leq \tau_h$ and $r_k^{(t)} < \tau_s$; BLACKLIST remains irreversible within an experiment. The relevant counters reset when their condition fails, and the state update finishes before layer aggregation. First-round control and permanent admission decisions consequently have different semantics: the former limits the current contribution, while the latter requires persistence.

The two streams meet only when computing aggregation privilege:

$$RawScore_k^{(t)} = (TrustScore_k^{(t)})^\alpha (ContentScore_k^{(t)})^\beta (HistPerf_k^{(t)})^\gamma \times (1 - RiskEMA_k^{(t)})^p. \quad (10)$$

The resulting state machine is shown in Fig. 4. Consider a client that contributes normally before submitting a harmful update: its historical utility may remain high, but current probes can lower its aggregation privilege or trigger the specified quarantine rule before aggregation. A benign long-tail client follows a different path. Weak content quality may lead to recoverable review, while permanent blacklisting requires persistent high risk. Recovery still depends on the stated consecutive-round utility and risk conditions, so retention of history provides an opportunity to recover rather than automatic readmission.

4.3 Risk-Gated Layered Aggregation

Backdoor effects may concentrate in deeper layers even when shallow features retain useful heterogeneous information. A uniform clipping rule then faces a

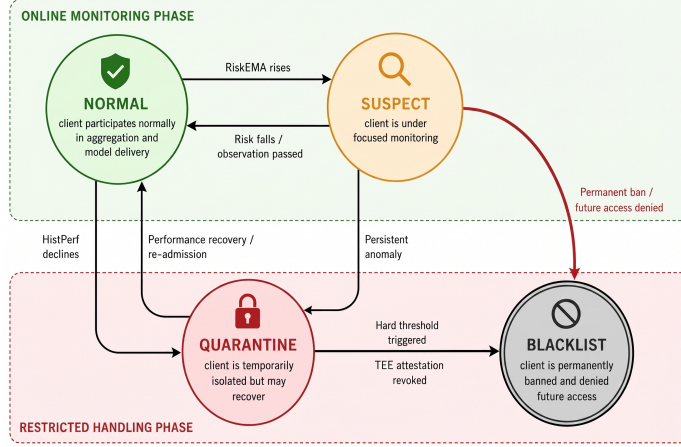


Fig. 4. Client state-transition automaton driven by historical utility and short-term risk.

tradeoff between suppressing the affected layers and preserving contributions elsewhere. Figure 5 shows how Trust Flow addresses this through layer-specific sensitivity, entry thresholds, and clipping radii. With L total layers, each layer l uses three normalized components:

$$\begin{aligned}
 S_{priv}^{(l)} &= \exp(-\tau_p l / L), \quad S_{util}^{(l)} = \frac{\|g_{root}^{(l)}\|_2}{\max_m \|g_{root}^{(m)}\|_2 + \varepsilon}, \\
 S_{sec}^{(l)} &= \frac{1}{2} \left(1 - \frac{1}{|\mathcal{T}|} \sum_{k \in \mathcal{T}} \cos(\Delta W_k^{(l)}, g_{root}^{(l)}) \right).
 \end{aligned} \tag{11}$$

Here $\mathcal{T} = \{k \in \mathcal{A} : M_{attest,k} = 1, RiskEMA_k^{(t-1)} < \tau_s\}$ is the trusted reference set. If it is empty, the admitted set $\mathcal{A}$ is used only to normalize the layer score, and the absence of a trusted peer reference is recorded. Thus $S_{priv}^{(l)}, 1 - S_{util}^{(l)}, S_{sec}^{(l)} \in [0, 1]$. The implementation sets $S_{total}^{(l)} = w_p S_{priv}^{(l)} + w_u (1 - S_{util}^{(l)}) + w_s S_{sec}^{(l)}$, with $w_p, w_u, w_s = (0.3, 0.3, 0.4)$ and $w_p + w_u + w_s = 1$, so $S_{total}^{(l)} \in [0, 1]$. The threshold and clipping radius are

$$\theta^{(l)} = \mu_{base} + \lambda_s S_{total}^{(l)}, \quad C^{(l)} = \frac{C_{base}}{S_{total}^{(l)} + \varepsilon_c}. \tag{12}$$

A layer showing stronger suspicious drift therefore has a higher entry threshold and a smaller clipping radius. Layers that remain close to the reference retain more capacity for benign heterogeneity.

The settings are $\tau_p = 3.0$ and $\varepsilon = \varepsilon_c = 10^{-8}$. C_{base} is the median trusted-client update norm before layer gating. The sensitivity terms remain dimensionless, and the clipping scale follows the update magnitude of each round.

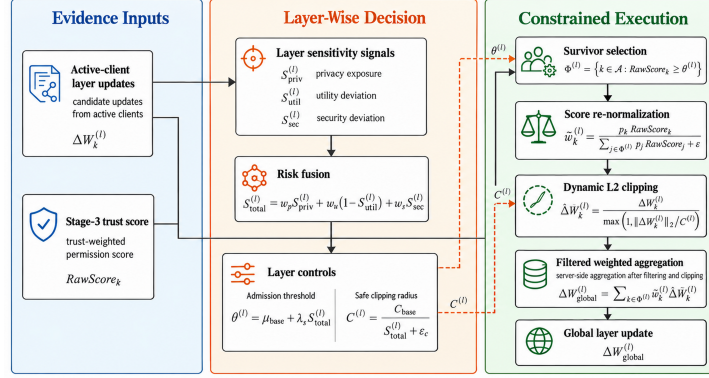


Fig. 5. Risk-gated layer-wise clipping and differentiated aggregation.

The surviving clients are $\Phi^{(l)} = \{k \in \mathcal{A} \mid RawScore_k^{(t)} \geq \theta^{(l)}\}$. Their weights are re-normalized:

$$\tilde{w}_k^{(l)} = \frac{p_k RawScore_k^{(t)}}{\sum_{j \in \Phi^{(l)}} p_j RawScore_j^{(t)} + \epsilon}, \quad \widehat{\Delta W}_k^{(l)} = \frac{\Delta W_k^{(l)}}{\max(1, \|\Delta W_k^{(l)}\|_2 / C^{(l)})}. \quad (13)$$

The final layer update is $\Delta W_{global}^{(l)} = \sum_{k \in \Phi^{(l)}} \tilde{w}_k^{(l)} \widehat{\Delta W}_k^{(l)}$. Re-normalization keeps the effective learning rate from shrinking merely because risky clients were removed.

If $\Phi^{(l)} = \emptyset$ because a sensitive layer rejects every candidate, the server keeps $k^* = \arg \max_{k \in \mathcal{A}} RawScore_k^{(t)}$ for that layer, gives it unit normalized weight, and still applies $C^{(l)}$. The layer update remains defined, while the threshold for other clients is unchanged.

Layer-wise control allows the same client update to receive different treatment across the network. When a semantic backdoor mainly alters deep features, sensitivity-dependent thresholds and clipping can constrain those layers while allowing more of a consistent shallow-layer update to contribute. Re-normalization addresses a separate optimization effect: removing clients also removes aggregation weight, which would otherwise reduce the update magnitude merely because filtering occurred. Projecting the data-size-weighted surviving scores back to a valid aggregation simplex separates that weight loss from the intended clipping operation.

4.4 Risk Probe Computation

The short-term risk stream uses several intentionally redundant probes. For update k in round t , the normalized channels are

$$r_{grad,k}^{(t)} = \text{clip}\left(\frac{1 - \cos(\Delta W_k, g_{root})}{2}, 0, 1\right), \quad (14)$$

$$r_{amp,k}^{(t)} = \text{clip}\left(\frac{\|\Delta W_k\|_2}{\text{median}_j \|\Delta W_j\|_2 + \varepsilon} - 1, 0, 1\right), \quad (15)$$

$$r_{clean,k}^{(t)} = \text{clip}\left(\frac{\mathcal{L}_{probe}(W + \Delta W_k) - \mathcal{L}_{probe}(W)}{\delta_{clean} + \varepsilon}, 0, 1\right), \quad (16)$$

$$r_{trigger,k}^{(t)} = \text{clip}\left(\frac{\text{KL}(\mathcal{H}(A_{base})\|\mathcal{H}(A_k))}{\delta_{trigger} + \varepsilon}, 0, 1\right), \quad (17)$$

$$\bar{c}_k^{(t)} = \frac{\sum_{j \neq k} \text{TrustScore}_j c_{kj}}{\sum_{j \neq k} \text{TrustScore}_j + \varepsilon}, \quad r_{peer,k}^{(t)} = \text{clip}(1 - \bar{c}_k^{(t)}, 0, 1), \quad (18)$$

Here $c_{kj} = \cos(\Delta W_k, \Delta W_j)$. For the trigger probe, A_{base} is the activation histogram of the current global model on the server-held proxy set, and A_k is the histogram after candidate update k is applied. The histograms use 32 bins from the last convolutional block and the pre-classifier representation. In each round, δ_{clean} and $\delta_{trigger}$ equal the median plus three median absolute deviations of the corresponding values over $\mathcal{T}$, so calibration is independent of the candidate under test. Define $Risk_k^{inst} = \max\{1 - M_{attest,k}, r_{grad,k}, r_{amp,k}, r_{clean,k}, r_{trigger,k}, r_{peer,k}\}$.

The maximum rule combines probes whose weaknesses differ. Direction and amplitude checks can expose explicit parameter attacks, but a backdoor may preserve those statistics while changing clean loss or internal activations. The clean-loss and activation channels provide additional evidence in that case. Peer disagreement remains supporting evidence because benign long-tail updates can also differ from the majority. After reference construction, the server audits the current update, advances the state, and assigns layer privileges, allowing the strongest warning to enter the participation decision without requiring every probe to agree.

4.5 Assurance Composition and Reference Role

The evidence sources complement one another because each resolves a different uncertainty. Attestation and privacy-aware verification establish execution provenance and bounded statistics for admission, while cloud auditing examines the submitted update against reference and peer evidence. Temporal states retain the distinction between a transient deviation and persistent risk. Passing the measured execution path therefore leads into semantic auditing; it does not bypass it. Likewise, low similarity can reduce contribution without serving as sufficient evidence for permanent exclusion.

The clean proxy supplies a stable initial direction and calibrates the probe interface when a small server-held set is available. Its role is embedded in the wider decision process: reference agreement contributes to content quality, while peer evidence, historical contribution, and recent risk still determine participation. When no proxy is available, the peer-reference branch constructs the guiding direction from admitted updates and follows the same content and temporal sequence. The reference is thus an input to auditing, with final privileges determined by state transitions and layer-wise aggregation.

4.6 From Separate States to Participation Decisions

The distinction between historical contribution and recent risk matters most for clients near a participation boundary. A client with low utility but low risk may receive reduced influence or reversible review, allowing later evidence to clarify whether its deviation was benign. A client with substantial past utility and rising risk instead faces restrictions despite that history. Persistent high risk is required for irreversible blacklisting, so the same threshold need not serve simultaneously as a contribution filter and a permanent-exclusion rule.

For fixed admission factor, content score, and historical utility, write $B_k = (TrustScore_k)^\alpha (ContentScore_k)^\beta (HistPerf_k)^\gamma$. With $p > 0$ and $0 \leq r < 1$, the score satisfies

$$\frac{\partial[B_k(1-r)^p]}{\partial r} = -pB_k(1-r)^{p-1} \leq 0, \quad r = RiskEMA_k^{(t)}. \quad (19)$$

At $r = 1$, the score is zero. This statement concerns the unnormalized score with the other factors fixed; layer selection, fallback, and normalization determine the update that is eventually applied. SUSPECT and QUARANTINE clients recover only under the stated consecutive-round utility and risk conditions. Low similarity alone is therefore not interpreted as maliciousness.

The mechanism-level evidence is separated across the ablations. Table 3 compares single-stream variants, HistPerf-only, and the complete dual stream on false bans and detection. Figure 7 isolates probe contributions and re-normalization. The first comparison tests the temporal policy; the second tests the evidence supplied to it and the optimization behavior after filtering.

5 Experiments

5.1 Setup

Experiments use Flower 1.5.0 and PyTorch on CIFAR-10 with a CIFAR-10-adapted ResNet-18. The first convolution uses a 3×3 kernel with stride one, the initial max-pooling layer is removed, and the classifier has ten outputs. Training is evaluated on paired host configurations, with the client execution path deployed in an Intel TDX confidential virtual machine. NVIDIA A10 GPUs execute model training. The TDX guest protects attestation, trusted monitoring,

report generation, and admission-state processing. The TDX procedure obtains a TDREPORT, converts it into a remotely verifiable Quote, checks nonce freshness and measured-code identity, and binds the round identifier and update digest before admission. The testbed uses anonymized paired host configurations and five NVIDIA A10 GPUs. The software stack is Ubuntu 20.04.6 LTS and CUDA 12.8. The 50,000 training images are partitioned into a shared 5,000-image client pool, replicated as a common base subset for all clients, and 45,000 exclusive images. Clients 0–9 use an IID split, clients 10–14 use Dirichlet $\alpha = 1.0$, and clients 15–19 use Dirichlet $\alpha = 0.1$. Separately, the server holds a balanced 500-image proxy set, with 50 images per class, from the held-out CIFAR-10 reference partition. It is disjoint from client training and final evaluation. The fixed proxy partition and initialization are shared across all methods. Reference-dependent methods use it for their defined guiding operation and probes, while methods without a reference channel retain their standard aggregation rule. All methods use the same optimizer, communication budget, client partition, and initial model. Five independent seeds are run on each hardware configuration, and means and standard deviations are computed over the resulting ten platform-seed runs.

The formal 30% attack setting uses six malicious clients. Client 0 performs untargeted label flipping with $\gamma = 0.5$. Client 1 injects a fixed 3×3 lower-right trigger into 20% of local samples and targets class 0. Client 2 uses label-preserving clean-label perturbations on 20% of local samples with the same target. Client 3 applies a fixed semantic attribute bias to 20% of local samples, with the biased subset assigned to class 0. Client 4 applies untargeted sign flipping, and Client 5 applies untargeted gradient scaling by a factor of 100. Clients 1–3 are evaluated as targeted attacks, whereas Clients 0, 4, and 5 are evaluated through clean loss, convergence, and detection state. Attack behaviors are active in every local training round; no attack is introduced only after observing the global trajectory. Five Sybil nodes without valid certificates and three free-rider nodes with forged low workload are used only to validate Phase-One gating. These eight nodes are intercepted before statistical aggregation, so later metrics are computed over the 20 admitted clients. Malicious updates that pass hardware attestation represent an attacker operating inside an admissible training process; post-attestation host tampering is excluded by the stated threat model. Each run uses 30 rounds and three local epochs. Aggregate values are averaged over the pooled platform-seed runs; state trajectories use one fixed seed per platform.

Clean accuracy and targeted ASR are reported as mean and standard deviation over the ten platform-seed runs. For each targeted behavior $a \in \{\text{trigger, clean, semantic}\}$, let $\mathcal{B}_a$ be its trigger- or attribute-bearing test subset and y_a^* its declared target. Define $ASR_a = |\{x \in \mathcal{B}_a : \arg \max f(x) = y_a^*\}|/|\mathcal{B}_a|$ and report the macro-average $ASR_{target} = \frac{1}{3} \sum_a ASR_a$. The three attack-specific quantities remain separate in the convergence traces; the headline table uses the macro-average so that no targeted behavior receives extra weight. Untargeted label/sign/scaling behavior is assessed through clean accuracy, loss, and state-transition time rather than this numerator. Permanent-ban FPR counts only benign clients that are irreversibly blacklisted; temporary SUSPECT and

Table 1. Experimental configuration and main hyperparameters.

Category	Setting	Value
FL task	Dataset/model/admitted clients	CIFAR-10/CIFAR-adapted ResNet-18/ $K = 20$
Optimization	Local epochs, lr, momentum, rounds	3, 0.001, 0.9, 30
Data heterogeneity	Shared client pool; server proxy; Group A/B/C	5000; 500; $\alpha = 1.0, 0.1$
Trust streams	$\alpha_1, \alpha_2, \beta_{fusion}; \beta_h, \beta_r$	0.6, 0.4, 2.0; 0.9, 0.85
Admission filter	$w_a, \lambda, Q; T_0, P_0, RiskEMA_0$	0.7, 1.0, 0.01; 1, 0.25, 0
Layer gating	$\alpha, \beta, \gamma, p; \mu_{base}, \lambda_s$	3.0, 1.0, 0.5, 1.0; 0.4, 1.2
Sensitivity	$\tau_p; w_p, w_u, w_s$	3.0; 0.3, 0.3, 0.4
Numerical stability	$\varepsilon, \varepsilon_c; C_{base}$	$10^{-8}, 10^{-8}$; median trusted norm
State machine	$\tau_h, \tau_s, \tau_q, \tau_b$; persistence	0.26, 0.64, 0.80, 0.90; 2, 3, 4 rounds

QUARANTINE states are recoverable controls. Stateless baselines have no comparable permanent-ban state, so their FPR is reported as N/A. All methods share the same client partition, initial model, optimizer, communication budget, attack schedule, reference availability, and Non-IID split; each applies its own defined decision rule to the shared evidence.

5.2 Overall Defense Performance

Figure 6 collects convergence, the three attack-specific ASR traces, trust-state trajectories, and feature separation. In the stated 30% mixed-attack protocol, all six attack behaviors run concurrently in every local round; the experiment is therefore a simultaneous-attack test. Trust Flow reaches 92.31% clean accuracy and 10.21% macro-averaged targeted ASR, close to the 10% random-target rate for ten CIFAR-10 classes. The state traces show the division of labor among the assurance layers. Execution-valid clients remain eligible for semantic auditing, current probes expose harmful update behavior, and the temporal streams determine whether an observation is transient or persistent. A legitimate long-tail client may receive a provisional restriction while retaining its history, whereas persistent risk drives the specified state transitions. These measurements describe the stated protocol and the evidence interfaces shared by the compared methods.

The trajectories in Fig. 6(b) connect the aggregate results to client-level decisions. Elevated risk is retained for label, trigger, semantic, sign, and scaling attacks, whereas a benign long-tail client’s weak contribution need not produce the sustained risk required for blacklisting. Its subsequent recovery depends on the specified utility and risk conditions. Figure 6(c) visualizes the resulting trust-state features through t-SNE; accuracy, ASR, and state-transition metrics provide the quantitative basis for assessing the defense.

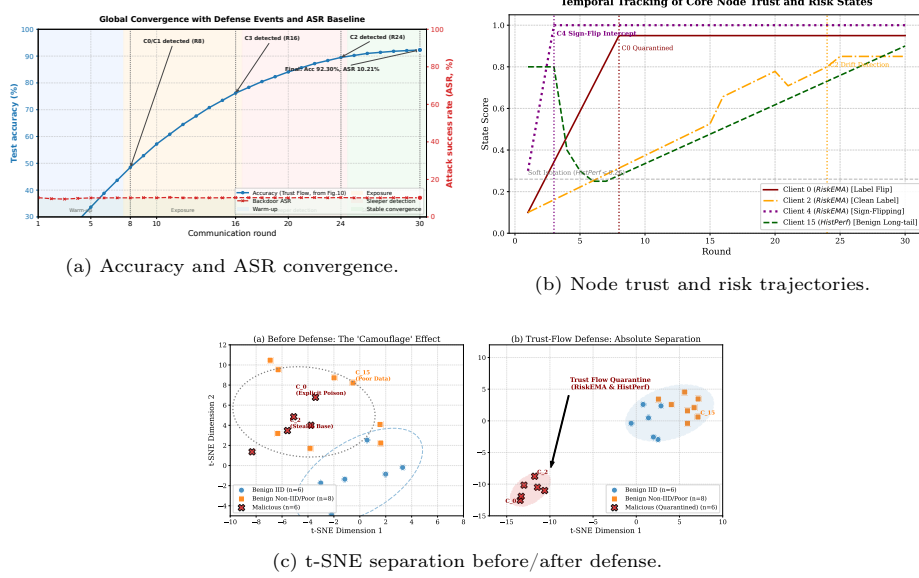


Fig. 6. Main defense behavior under mixed attacks.

In the matched classical comparison, Krum reaches 64.20% accuracy, Trimmed Mean 71.35%, and FLTrust 81.25%, while Trust Flow reaches 92.31%. These values do not establish a universal ranking. DeepSight, CrowdGuard, RoseAgg, FLPurifier, ShieldFL, and RaSA require attack-specific or privacy-preserving interfaces that are not reproduced here, so their absence is a comparison limitation. The 50% result remains an out-of-assumption stress test rather than evidence of a theoretical Byzantine boundary.

5.3 Ablation, Sensitivity, and Overhead

Table 3 examines whether one state can serve both retention and detection. The aggressive single-stream variant attains high recall at the cost of more permanent false positives, while the conservative variant reduces false positives but misses more malicious clients. HistPerf-only further illustrates the limitation of relying on contribution history: it retains benign clients but provides little attack sensitivity. The dual-stream result supports assigning those roles to separate states. The reported TrustReport size is about 15 KB, and server aggregation plus audit increases from 2.10 s to 4.85 s. Optional cryptographic secure aggregation remains outside the stated system boundary.

The server timing covers aggregation and content audit. The measured auxiliary security-path overhead is approximately 11 ms per admitted-client round. Device-side TEE Quote generation, signing, and lightweight monitoring are measured separately at approximately 12–15 ms per admitted-client round. These values describe their respective measurement scopes rather than one end-to-end

Table 2. Performance comparison under 30% mixed attacks and boundary stress testing.

Method/Setting	Core mechanism	Acc.	Targeted ASR	Perm. FPR [†]
FedAvg [2]	no defense	86.15 $\pm$ 1.20	92.34 $\pm$ 5.10	N/A
Krum [3]	Euclidean selection	64.20 $\pm$ 2.80	12.15 $\pm$ 0.90	N/A
Trimmed Mean [4]	coordinate trimming	71.35 $\pm$ 2.40	38.60 $\pm$ 4.20	N/A
FLTrust [5]	clean anchor weighting	81.25 $\pm$ 1.50	15.42 $\pm$ 1.10	N/A
FLAME [10]	clustering, clipping, noise	73.18 $\pm$ 1.80	19.00 $\pm$ 0.60	N/A
FLDetector [8]	update consistency	65.89 $\pm$ 2.30	23.00 $\pm$ 1.20	N/A
FreqFed [22]	frequency analysis	79.37 $\pm$ 0.90	14.00 $\pm$ 1.10	N/A
Trust Flow	dual stream + layered gating	92.31 $\pm$ 0.09	10.21 $\pm$ 0.05	0.00
Boundary stress: 50% malicious	10/20 malicious	91.81 $\pm$ 0.10	10.46 $\pm$ 0.06	0.00

[†] Permanent-ban FPR is defined only for Trust Flow and its stateful ablations. Krum, Trimmed Mean, and FLTrust are stateless in this comparison and are therefore marked N/A. Temporary SUSPECT/QUARANTINE states are recoverable and reported separately through trajectories.

Table 3. Ablation and overhead summary.

Study	Variant / architecture	FPR or overhead	TPR / time
Reputation	Single-stream EMA, aggressive	18.25% FPR	95.12% TPR
Reputation	Single-stream EMA, conservative	2.10% FPR	48.33% TPR
Reputation	HistPerf-only	0.15% FPR	21.05% TPR
Reputation	Dual-stream Trust Flow	0.00% FPR	100.00% TPR
Overhead	Standard FedAvg	0 KB extra	2.10 s aggregation
Overhead	Trust Flow	$\sim$ 15 KB TrustReport	4.85 s incl. audit

deployment latency. CPU-side TDX isolation does not by itself establish protection of GPU execution or GPU memory.

Figures 7 and 8 show that treating SUSPECT as recoverable review yields 88.33% per-round malicious coverage and a 14.79% benign review rate, with no permanent false ban in the tested runs. Activation and clean-loss probes reduce the reported clean-label ASR below 9.2%; re-normalization avoids an implicit learning-rate decay. At $\alpha = 0.1$, Trust Flow remains near 92% accuracy across the tested Intel TDX configurations. These ablation values are descriptive measurements under the same protocol and do not claim statistical significance beyond the repeated-run summary.

The Dirichlet sweep in Fig. 8 holds label space, model, optimizer, and attack protocol fixed while varying data heterogeneity. As α decreases, benign updates become less aligned, making the filtering decisions of Krum and FLTrust more likely to reject legitimate drift. Trust Flow is less sensitive in the reported sweep because current content quality is considered together with historical utility and recent risk. The default $\beta_{fusion} = 2.0$ gives the best observed accuracy-ASR balance. The reported 2.10 s to 4.85 s runtime covers server-side aggregation and auditing, not end-to-end TDX deployment latency.

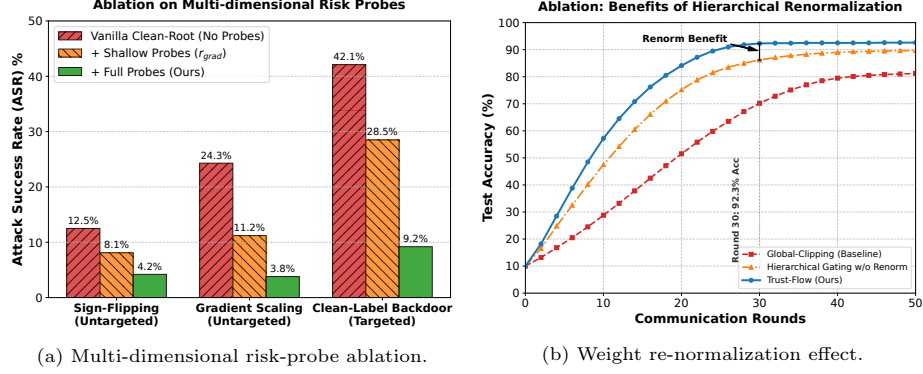


Fig. 7. Component-level ablation of risk probes and re-normalization.

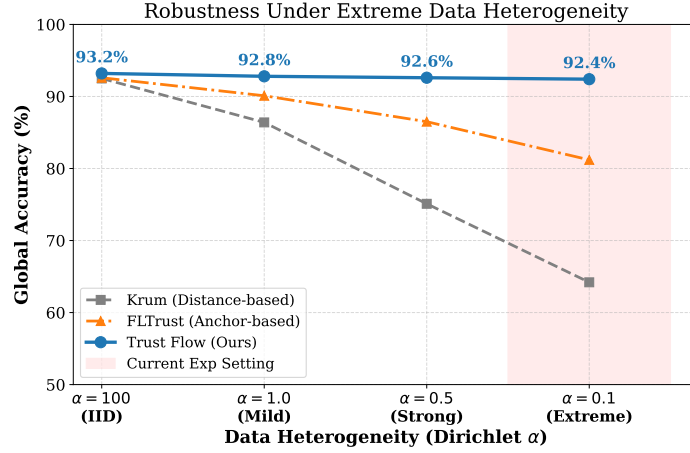


Fig. 8. Robustness stress test under varying data heterogeneity.

6 Discussion and Conclusion

Trust Flow connects execution admission, privacy-aware statistical checks, content auditing, and layer-wise aggregation through separate historical-utility and short-term-risk states. This separation addresses the tension between retaining benign long-tail contributions and restricting harmful updates from clients with favorable histories. Execution and current-update evidence inform the states, while persistence rules distinguish recoverable restrictions from permanent exclusion. On the Intel TDX-backed execution path and paired host configuration, this separation keeps permanent-ban FPR near zero while retaining high recall against the tested mixed attacks. The results relate the participation policy to both attack suppression and benign-client retention.

The evaluated profile is server-assisted vision FL with a small trusted reference partition and an attestation-capable client path. The peer-reference branch follows the same decision sequence when no server proxy is available, while the measurements here use the common reference-initialized path. Microarchitectural side channels and confidential-VM resource contention remain outside the threat model. The probes are tailored to vision classification; report size and server audit overhead are kept separate from end-to-end latency. Cryptographic secure aggregation is outside the main boundary because the server must inspect individual evidence. This profile states where the trust-flow contract applies and provides a basis for extending it to other data modalities and deployment conditions.

References

1. Kairouz, P., McMahan, H.B., Avent, B., et al.: Advances and open problems in federated learning. *Foundations and Trends in Machine Learning* 14(1–2), 1–210 (2021). <https://doi.org/10.1561/22000000083>
2. McMahan, B., Moore, E., Ramage, D., Hampson, S., Aguera y Arcas, B.: Communication-efficient learning of deep networks from decentralized data. In: *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*, PMLR 54, pp. 1273–1282 (2017). <https://proceedings.mlr.press/v54/mcmahan17a.html>
3. Blanchard, P., El Mhamdi, E.M., Guerraoui, R., Stainer, J.: Machine learning with adversaries: Byzantine tolerant gradient descent. In: *Advances in Neural Information Processing Systems* 30 (2017). [NeurIPS paper page](#)
4. Yin, D., Chen, Y., Ramchandran, K., Bartlett, P.: Byzantine-robust distributed learning: Towards optimal statistical rates. In: *Proceedings of the 35th International Conference on Machine Learning*, PMLR 80, pp. 5650–5659 (2018). <https://proceedings.mlr.press/v80/yin18a.html>
5. Cao, X., Fang, M., Liu, J., Gong, N.Z.: FLTrust: Byzantine-robust federated learning via trust bootstrapping. In: *Proceedings 2021 Network and Distributed System Security Symposium (NDSS)* (2021). <https://doi.org/10.14722/ndss.2021.24434>
6. Fung, C., Yoon, C.J.M., Beschastnikh, I.: The limitations of federated learning in Sybil settings. In: *23rd International Symposium on Research in Attacks, Intrusions and Defenses (RAID 2020)*, pp. 301–316. USENIX Association (2020). <https://www.usenix.org/conference/raid2020/presentation/fung>
7. Wang, L., Huang, M., Zhang, Z., Li, M., Wang, J., Gai, K.: RaSA: Robust and adaptive secure aggregation for edge-assisted hierarchical federated learning. *IEEE Transactions on Information Forensics and Security* 20, 4280–4295 (2025). <https://doi.org/10.1109/TIFS.2025.3559411>
8. Zhang, Z., Cao, X., Jia, J., Gong, N.Z.: FLDetector: Defending federated learning against model poisoning attacks via detecting malicious clients. In: *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pp. 2545–2555 (2022). <https://doi.org/10.1145/3534678.3539231>
9. Rieger, P., Nguyen, T.D., Miettinen, M., Sadeghi, A.R.: DeepSight: Mitigating backdoor attacks in federated learning through deep model inspection. In: *Network and Distributed System Security Symposium (NDSS)* (2022). <https://doi.org/10.14722/ndss.2022.23156>

10. Nguyen, T.D., Rieger, P., Chen, H., et al.: FLAME: Taming backdoors in federated learning. In: 31st USENIX Security Symposium, pp. 1415–1432 (2022). <https://www.usenix.org/conference/usenixsecurity22/presentation/nguyen>
11. Zhang, J., Zhu, C., Sun, X., Ge, C., Chen, B., Susilo, W., Yu, S.: FLPurifier: Backdoor defense in federated learning via decoupled contrastive training. *IEEE Transactions on Information Forensics and Security* 19, 4752–4766 (2024). <https://doi.org/10.1109/TIFS.2024.3384846>
12. Rieger, P., Krauß, T., Miettinen, M., Dmitrienko, A., Sadeghi, A.R.: CrowdGuard: Federated backdoor detection in federated learning. In: Network and Distributed System Security Symposium (NDSS) (2024). <https://doi.org/10.14722/ndss.2024.23233>
13. Yang, H., Xi, W., Shen, Y., Wu, C., Zhao, J.: RoseAgg: Robust defense against targeted collusion attacks in federated learning. *IEEE Transactions on Information Forensics and Security* 19, 2951–2966 (2024). <https://doi.org/10.1109/TIFS.2024.3352415>
14. Ma, Z., Ma, J., Miao, Y., Li, Y., Deng, R.H.: ShieldFL: Mitigating model poisoning attacks in privacy-preserving federated learning. *IEEE Transactions on Information Forensics and Security* 17, 1639–1654 (2022). <https://doi.org/10.1109/TIFS.2022.3169918>
15. Chen, Y., Luo, F., Li, T., Xiang, T., Liu, Z., Li, J.: A training-integrity privacy-preserving federated learning scheme with trusted execution environment. *Information Sciences* 522, 69–79 (2020). <https://doi.org/10.1016/j.ins.2020.02.037>
16. Xu, T., Zhu, K., Andrzejak, A., Zhang, L.: Distributed learning in trusted execution environment: A case study of federated learning in SGX. In: 2021 7th IEEE International Conference on Network Intelligence and Digital Content (IC-NIDC), pp. 450–454 (2021). <https://doi.org/10.1109/IC-NIDC54101.2021.9660433>
17. Zhang, X., Chen, Z., Chen, G., Feng, X., Shen, Q., Wu, Z.: RPPFL: Robust and privacy-preserving federated learning via trusted execution environments. In: ICASSP 2025 - 2025 IEEE International Conference on Acoustics, Speech and Signal Processing, pp. 1–5 (2025). <https://doi.org/10.1109/ICASSP49660.2025.10889398>
18. Queyrut, S., Schiavoni, V., Felber, P.: Mitigating adversarial attacks in federated learning with trusted execution environments. In: 2023 IEEE 43rd International Conference on Distributed Computing Systems (ICDCS), pp. 626–637 (2023). <https://doi.org/10.1109/ICDCS57875.2023.00069>
19. Lu, Z., Wang, L., Zhang, Z., Huang, M., Wang, J., Li, M.: TMT-FL: Enabling trustworthy model training of federated learning with malicious participants. *IEEE Transactions on Dependable and Secure Computing* 22(3), 2723–2740 (2025). <https://doi.org/10.1109/TDSC.2024.3521377>
20. Bagdasaryan, E., Veit, A., Hua, Y., Estrin, D., Shmatikov, V.: How To Backdoor Federated Learning. In: Proceedings of the Twenty Third International Conference on Artificial Intelligence and Statistics, PMLR 108, pp. 2938–2948 (2020). <https://proceedings.mlr.press/v108/bagdasaryan20a.html>
21. Wang, B., Yao, Y., Shan, S., Li, H., Viswanath, B., Zheng, H., Zhao, B.Y.: Neural Cleanse: Identifying and mitigating backdoor attacks in neural networks. In: 2019 IEEE Symposium on Security and Privacy (SP), pp. 707–723 (2019). <https://doi.org/10.1109/SP.2019.00031>
22. Fereidooni, H., Pegoraro, A., Rieger, P., Dmitrienko, A., Sadeghi, A.R.: FreqFed: A frequency analysis-based approach for mitigating poisoning attacks in federated learning. In: Network and Distributed System Security Symposium (NDSS) (2024). <https://doi.org/10.14722/ndss.2024.23620>